Introduction to Python Theory:

1)Introduction to Python and its Features (simple, high-level, interpreted language).

Ans:- Python is a simple, easy-to-learn programming language used for web development, data analysis, artificial intelligence, and more. It is popular because its syntax is clear and similar to everyday English.

Key Features of Python

- **Simple and easy**: Code is readable and beginner-friendly.
- **High-level language**: You don't need to manage hardware details like memory.
- **Interpreted**: Code runs line by line, making debugging easier.
- **Portable**: Runs on different systems (Windows, Mac, Linux).
- **Large library support**: Comes with many built-in modules and tools.
- **Versatile**: Used in many fields like web apps, automation, and data science.

2) History and evolution of Python.

Ans:- Python was created by Guido van Rossum in the late 1980s and first released in 1991. It was designed to be simple, readable, and easy to use.

- **1991**: First version of Python released
- **2000 (Python 2.0)**: Added new features like list comprehensions and better memory management
- **2008 (Python 3.0)**: Major update with improved design and removal of older features
- **Today**: Python continues to grow, widely used in web development, AI, data science, and automation

3)Advantages of using Python over other programming languages.

Ans:- **Advantages of Using Python**

- **Easy to learn**: Simple and readable syntax, great for beginners
- **High-level**: No need to manage complex system details
- **Interpreted**: Easier testing and debugging
- **Large library support**: Many built-in modules and frameworks
- **Cross-platform**: Runs on Windows, Mac, and Linux

- **Versatile**: Used in web development, data science, AI, and automation
- **Strong community support**: Lots of tutorials and help available

4)Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).

Ans:- **1. Install Python**

- Go to the official website and download Python
- Run the installer and check **"Add Python to PATH"**
- Verify installation using python --version in terminal

2. Using Anaconda

- Download and install Anaconda
- Comes with Python, libraries, and Jupyter Notebook
- Use Anaconda Navigator to manage environments easily

3. Using PyCharm

- Download and install PyCharm
- Create a new project and select Python interpreter
- Provides smart coding, debugging, and tools

4. Using Visual Studio Code

- Install VS Code
- Add the Python extension
- Select Python interpreter and start coding

5) Writing and executing your first Python program.

Ans:- **1. Write the program**

Open a text editor and type:

```
print("Hello, World!")
```

Save the file as: hello.py

2. Run the program

Open Command Prompt (Windows) or Terminal (Mac/Linux), then type:

```
python hello.py
```

(or use python3 hello.py if needed)

3. Output

You'll see:

Hello, World!

6) Understanding Python's PEP 8 guidelines.

Ans:- PEP 8 is the official style guide for Python code. It helps you write code that is clean, readable, and consistent.

1. Indentation

Use **4 spaces** per indentation level (don't use tabs).

```
if True:
    print("Hello")
```

2. Line Length

Keep each line under **79 characters**.

3. Naming Style

- Variables & functions → lowercase_with_underscores
- Classes → CapitalizedWords

```
my_variable = 10

def my_function():
    pass

class MyClass:
    pass
```

4. Spaces

Use spaces around operators and after commas:

```
x = 5 + 3
print(x, y)
```

5. Blank Lines

- Use blank lines to separate functions and classes
- Keeps code easy to read

6. Imports

Write imports at the top, one per line:

```
import math
import os
```

7. Comments

Write clear, short comments:

```
# This prints a message
print("Hello")
```

7) Indentation, comments, and naming conventions in Python.

Ans:- **1. Indentation**

- Python uses indentation (spaces) to define blocks of code
- Use **4 spaces** (standard)

```
if True:
    print("Hello")
```

2. Comments

- Comments explain code and are ignored by Python
- Use # for single-line comments

```
# This prints a message
print("Hello")
```

3. Naming Conventions

- Variables & functions → lowercase_with_underscores
- Classes → CapitalizedWords

```
my_variable = 10

def my_function():
    pass

class MyClass:
    pass
```

8) **Writing readable and maintainable code.**

Ans:- Writing **readable and maintainable code** means making your code easy to understand, fix, and update later.

1. Use clear names

- Choose meaningful variable and function names
  (total_marks instead of tm)

2. Keep code simple

- Avoid unnecessary complexity
- Break big tasks into small functions

3. Follow consistent style

- Use proper indentation (4 spaces)
- Follow naming rules (like lowercase_with_underscores)

4. Add comments (when needed)

- Explain *why*, not obvious things

```
# Calculate total marks
def calculate_total(a, b):
    return a + b
```

5. Organize code

- Group related code together
- Use blank lines for clarit

9)Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.

Ans:- **1. Integer (int)**

Whole numbers

x = 10

2. Float

Decimal numbers

y = 3.5

3. String (str)

Text inside quotes

name = "Python"

4. List

Ordered, changeable collection

fruits = ["apple", "banana", "mango"]

5. Tuple

Ordered, but **cannot be changed**

colors = ("red", "green", "blue")

6. Dictionary (dict)

Key-value pairs

student = {"name": "Aman", "age": 20}

7. Set

Unordered, no duplicates

nums = {1, 2, 3, 3}

10)Python variables and memory allocation.

Ans:- **1. Variables**

A variable is just a **name that stores data**.

```
x = 10
name = "Python"
```

2. Memory allocation (simple idea)

When you create a variable, Python:

- Stores the **value in memory**
- Makes the variable name **point to that memory**

```
a = 10
b = a
```

11) Python operators: arithmetic, comparison, logical, bitwise.

Ans:- **1. Arithmetic Operators (math)**

Used for calculations:

- + add
- - subtract

- * multiply
- / divide
- % remainder

x = 10 + 5

2. Comparison Operators (compare values)

- == equal
- != not equal
- > greater than
- < less than

10 > 5 # True

3. Logical Operators (combine conditions)

- and → both true
- or → one true
- not → reverse

True and False # False

4. Bitwise Operators (work on bits)

- & AND
- | OR
- ^ XOR
- ~ NOT

5 & 3

12)Introduction to conditional statements: if, else, elif.

Ans:- **Conditional statements** in Python are used to make decisions based on conditions.

1. if

Runs code when condition is **true**

```
x = 10
if x > 5:
   print("Greater than 5")
```

2. else

Runs when condition is **false**

```
x = 3
if x > 5:
   print("Big")
else:
   print("Small")
```

3. elif

Checks **multiple conditions**

```
x = 7
if x > 10:
   print("Large")
elif x > 5:
   print("Medium")
else:
   print("Small")
```

13) Nested if-else conditions.

Ans:- n**ested if-else** means placing an **if-else inside another if-else**.

It is used when you need to check **multiple levels of conditions**.

Simple example:

```
x = 10

if x > 0:
   if x % 2 == 0:
      print("Positive even number")
   else:
      print("Positive odd number")
else:
   print("Negative number")
```

14)Introduction to for and while loops.

Ans:- **Loops** are used to repeat a block of code multiple times.

1. for loop

Used when you know how many times to repeat.

```
for i in range(5):
    print(i)
```

→ Prints numbers from 0 to 4

2. while loop

Used when repetition depends on a condition.

```
x = 1
while x <= 5:
    print(x)
    x += 1
```

→Runs until condition becomes false

15) How loops work in Python.

Ans:- **Loops in Python** are used to **repeat a block of code** multiple times without writing it again.

How loops work:

1. for loop (fixed repetition)

- Runs for a **specific number of times**
- Works with sequences like range, list, etc.

```
for i in range(3):
    print(i)
```

→ Step-by-step:

- i = 0 → run code
- i = 1 → run code
- i = 2 → run code
- stop

2. while loop (condition-based repetition)

- Runs **as long as condition is true**

```
x = 1
while x <= 3:
    print(x)
    x += 1
```

→ Step-by-step:

- check condition
- run code
- update value
- repeat until condition is false

16)Using loops with collections (lists, tuples, etc.).

Ans:- **Using loops with collections in Python** means repeating actions for items inside lists, tuples, sets, or dictionaries.

1. Loop with list

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
    print(fruit)
```

2. Loop with tuple

```
colors = ("red", "green", "blue")

for color in colors:
    print(color)
```

3. Loop with set

```
nums = {1, 2, 3}

for n in nums:
    print(n)
```

4. Loop with dictionary

```
student = {"name": "Aman", "age": 20}
```

```
for key, value in student.items():
    print(key, value)
```

17)Built-in methods for strings, lists, etc.

Ans:-Built-in methods are ready-made functions in Python that help you work with strings, lists, etc.

1. String methods

```
text = "hello"

text.upper()   # "HELLO"
text.lower()   # "hello"
text.replace("h", "j")  # "jello"
```

2. List methods

```
nums = [1, 2, 3]

nums.append(4)   # [1, 2, 3, 4]
nums.remove(2)   # [1, 3, 4]
nums.sort()      # [1, 3, 4]
```

3. Tuple methods

```
t = (1, 2, 2, 3)

t.count(2)   # 2
t.index(3)   # 3
```

4. Dictionary methods

```
student = {"name": "Aman", "age": 20}

student.keys()    # all keys
student.values()  # all values
student.items()   # key-value pairs
```

18) Understanding the role of break, continue, and pass in Python loops

Ans:- **1. break**

Stops the loop completely

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

→ Stops when i = 3

2. continue

Skips the current iteration and moves to next

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

→ Skips 3, prints others

3. pass

Does nothing (placeholder)

```
for i in range(5):
    if i == 3:
        pass
    print(i)
```

→Used when code is not written yet

19)Understanding how to access and manipulate strings.

Ans:- **1. Accessing strings (indexing)**

Each character has an index starting from 0.

```
text = "Python"

print(text[0])  # P
print(text[1])  # y
```

→ Negative index starts from the end:

```
print(text[-1])  # n
```

2. Slicing (getting part of string)

```
text = "Python"

print(text[0:3])  # Pyt
print(text[2:])   # thon
```

3. Common string methods (manipulation)

```
text = "hello world"

text.upper()       # "HELLO WORLD"
text.lower()       # "hello world"
text.replace("h", "j")  # "jello world"
text.strip()       # removes spaces
```

4. Joining strings

```
a = "Hello"
b = "World"

print(a + " " + b)  # Hello World
```

20)Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

Ans:- **1. Concatenation (joining strings)**

Use + to join strings.

```
a = "Hello"
b = "World"

print(a + " " + b)
```

→Output: Hello World

2. Repetition (repeat strings)

Use * to repeat a string.

```
text = "Hi "

print(text * 3)
```

→Output: Hi Hi Hi

3. String methods (changing text)

```
text = "hello world"

text.upper()   # "HELLO WORLD"
text.lower()   # "hello world"
text.title()   # "Hello World"
```

21)String slicing.

Ans:- **String slicing** in Python means getting a part (substring) of a string using indexes.

Syntax:

```
string[start:end:step]
```

1. Basic slicing

```
text = "Python"

print(text[0:3])   # Pyt
```

→Starts at index 0, ends before 3

2. Omit start or end

```
print(text[:4])   # Pyth
print(text[2:])   # thon
```

3. Negative indexing

```
print(text[-1])   # n
print(text[-3:])  # hon
```

4. Step value

```
print(text[0:6:2])  # Pto
```